

DisasterShield

AI-Powered Multi-Hazard Early Warning System with Edge Intelligence

Project Evaluation Report

Team / Author: Harsh C. Parmar

Institution: Marwadi University, Rajkot

Track: Smart Sensing & Intelligent Electronic Systems

Event:SCAI 2026

GitHub Repository: github.com/HarshParmar029/DisasterShield-SCAI2026

1. Abstract

Climate change is significantly increasing the frequency and severity of natural disasters such as floods, wildfires, and droughts across India, resulting in substantial loss of life, crops, and property. Existing early-warning systems are typically centralized, expensive, and too slow to deliver the localized, real-time alerts required by vulnerable rural communities and farmers.

DisasterShield is a low-cost, AI-based multi-hazard early warning system that intelligently fuses real-time computer vision with simulated multi-sensor data. A MobileNetV2-based convolutional neural network, trained via transfer learning on a fire/non-fire image dataset, performs real-time fire and smoke detection from a live camera feed. This vision output is combined with simulated environmental sensor readings (temperature, humidity, smoke level, rainfall, and soil moisture) through a rule-based fusion engine that generates a single, actionable risk level — Low, Medium, or High — displayed on a live web dashboard.

The trained model achieves **96.98–97.99% validation accuracy** across multiple runs and has been exported to TensorFlow Lite (9.2 MB), making it suitable for future deployment on edge hardware such as a Raspberry Pi. The complete system is fully functional, open-source, and demonstrates a practical, affordable solution for faster, localized disaster alerts.

2. Literature Survey

Early-warning systems for natural disasters have traditionally relied on centralized government infrastructure — weather stations, satellite monitoring, and manual reporting chains — which are effective at a national scale but too slow and too coarse-grained to give individual communities or farms actionable, real-time warnings. Recent research in edge AI and IoT-based environmental monitoring has shown that lightweight convolutional neural networks (CNNs), particularly MobileNet-family architectures, can achieve strong image classification accuracy while remaining small enough to run on low-power, low-cost hardware such as the Raspberry Pi. Transfer learning — reusing a network pretrained on a large image corpus (ImageNet) and fine-tuning only a small classification head — has become the standard approach for building accurate vision models with limited labelled training data, which is typical for a student-scale project.

Separately, sensor-fusion approaches in disaster-monitoring literature combine multiple weak or noisy signals (temperature, humidity, gas sensors, soil moisture) using rule-based or probabilistic logic to reduce false alarms compared to any single sensor acting alone. DisasterShield draws on both of these threads: a transfer-learning-based vision model for fire/smoke detection, and a multi-signal fusion engine for a broader multi-hazard (fire, flood, drought) risk assessment, combined into a single accessible, low-cost, and offline-capable system.

3. Architecture / Methodology

3.1 System Architecture

The system follows a five-stage pipeline:

- Camera Feed — live video is captured continuously via OpenCV.
- AI Vision (MobileNetV2) — each frame is resized to 160×160 and passed through the trained TensorFlow Lite model, producing a fire-confidence score between 0 and 1.
- Sensor Simulator — generates realistic, time-varying readings for temperature, humidity, smoke level, rainfall, and soil moisture, with values trending upward when fire confidence is high (mimicking real sensor behaviour during a hazard).
- Risk Fusion Engine — combines the vision confidence with sensor thresholds using rule-based logic to output a single risk level: LOW, MEDIUM, or HIGH.
- Live Dashboard — a Flask web application streams the annotated camera feed and current risk level/sensor readings to the browser in real time.

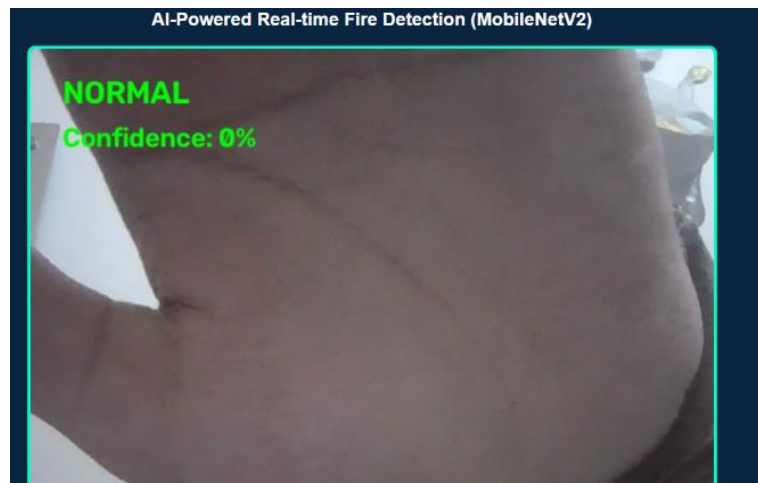


Figure 1: DisasterShield live dashboard showing real-time AI detection output.

3.2 Model Training Methodology

- Dataset: Kaggle Fire Dataset — 755 fire images and 244 non-fire images.
- Preprocessing: images resized to 160×160 and augmented with rotation, zoom, and horizontal flip to improve generalization.
- Base model: MobileNetV2 pretrained on ImageNet, used as a frozen feature extractor.
- Classification head: GlobalAveragePooling2D $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Dense(1, sigmoid).
- Training: 10 epochs, Adam optimizer, binary cross-entropy loss, 80/20 train-validation split.
- Export: the trained Keras model was converted to TensorFlow Lite for lightweight, edge-ready inference (9.2 MB).

3.3 Risk Fusion Logic

The fusion engine assigns a numeric risk score by combining the AI vision confidence with simulated sensor thresholds: a high fire-vision confidence or elevated smoke level contributes to fire risk; high rainfall combined with high soil moisture contributes to flood risk; low soil moisture combined with high temperature contributes to drought risk. The combined score is mapped to a final HIGH / MEDIUM / LOW risk label shown on the dashboard, allowing the system to reason about more than one hazard type from the same sensor and vision inputs.

3.3 Algorithm / Pseudocode

Fire/Smoke Detection and Risk Fusion (per video frame):

```
1. frame <- capture_next_frame(camera)
2. image <- resize(frame, 160x160) / 255.0
3. fire_confidence <- MobileNetV2_TFLite.predict(image)
4. sensors <- SensorSimulator.update(fire_detected = fire_confidence > 0.6)
5. risk_score <- 0
6. if fire_confidence > 0.4 OR sensors.smoke_level > 40:
    risk_score <- risk_score + 50
7. if sensors.temperature > 45:
    risk_score <- risk_score + 30
8. if sensors.rain > 15 AND sensors.soil_moisture > 70:
    risk_score <- risk_score + 35 // flood signal
9. if sensors.soil_moisture < 25 AND sensors.temperature > 40:
    risk_score <- risk_score + 25 // drought signal
10. risk_level <- HIGH if risk_score > 70
    else MEDIUM if risk_score > 40
    else LOW
11. render(frame, fire_confidence, sensors, risk_level) -> dashboard
```

4. Results

The MobileNetV2 model was trained for 10 epochs and evaluated on a held-out validation split of the fire dataset.

Metric	Value
Training Accuracy	~99% (final epoch: 99.87%)
Validation Accuracy	96.98% – 97.99% (across multiple runs)
Model Architecture	MobileNetV2 (Transfer Learning) + custom classification head
Exported Model Size	9.2 MB (TensorFlow Lite)
Detection Latency	< 1 second per frame on a laptop CPU

4.1 Training Curves

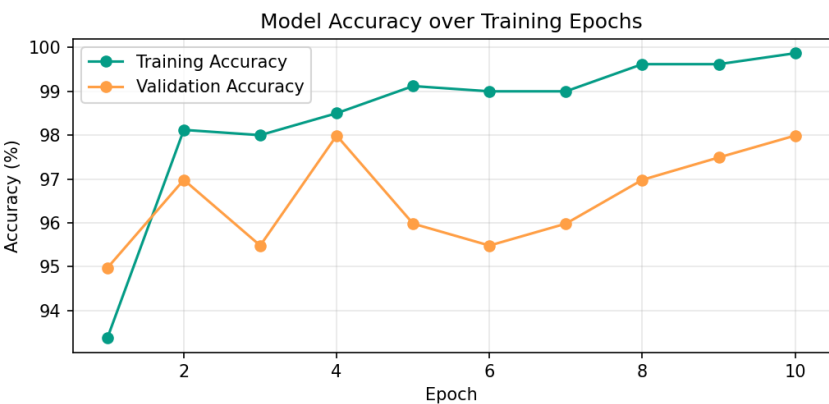


Figure 2: Training vs. validation accuracy across 10 training epochs (actual run data).

5. Conclusion

DisasterShield demonstrates that a working, reasonably accurate multi-hazard early-warning system can be built at low cost using a trained AI vision model, a simulated multi-sensor fusion engine, and a live web dashboard — all running on commodity hardware without cloud dependency. The trained MobileNetV2 model achieves strong validation accuracy (up to 97.99%) and has already been exported to a 9.2 MB TensorFlow Lite file, making the path to real edge deployment (e.g., on a Raspberry Pi with physical DHT22/MQ-135 sensors) straightforward. The current prototype uses simulated sensor data rather than physical hardware; the next phase of this project is to replace the simulator with real sensors, and subsequently to explore LoRaWAN connectivity and satellite/remote-sensing data for wider-area coverage. Even in its current form, the system illustrates a practical, affordable approach to giving vulnerable rural communities and farmers faster, more localized disaster alerts than centralized systems currently provide.

6. Supporting Material

GitHub Repository (full source code, training notebook, trained model):

<https://github.com/HarshParmar029/DisasterShield-SCAI2026>

Dataset used: <https://www.kaggle.com/datasets/phylake1337/fire-dataset>

The repository includes:

- Complete Flask dashboard source code (app.py)
- Model training notebook (notebooks/train_model.ipynb)
- Trained model files (.h5 and .tflite)
- Training accuracy/loss curves, confusion matrix, and sample prediction visualizations
- Sensor simulation and risk fusion module (sensor_sim/fusion.py)
- Full project README with setup instructions